**Informatik I**
Dr. Henrik Brosenne
Paul Sarnighausen-Cahn

**Wintersemester 2025/26**
Georg-August-Universität Göttingen
Institut für Informatik

# Übungsblatt 08

**Studentische Lehrveranstaltungsevaluation**

Die studentische Lehrveranstaltungsevaluation findet im Zeitraum

**14.01.2026, 03:00 Uhr bis 08.02.2026, 23:59 Uhr**

online in der Stud.IP-Veranstaltung **Informatik I (WiSe 2025/26)** statt.
Weitere Informationen zur zentralen Lehrveranstaltungsevaluation durch Studierende finden Sie unter:
`https://www.uni-goettingen.de/lehrveranstaltungsevaluation`

**Allgemeine Information zum Bestehen der Prüfungsvorleistung.**
Um die Prüfungsvorleistung zu bestehen, müssen Sie auf jedem Übungszettel mindestens 40% der Punkte in Theorie und Praxis erreichen. Es gibt jedoch zwei Ausnahmen: Sie dürfen auf zwei Zetteln bei den theoretischen Aufgaben und auf zwei Zetteln beim praktischen Teil unter die 40%-Grenze fallen, ohne dass dies zum Nichtbestehen führt.
Das bedeutet konkret: Wenn Sie beispielsweise auf einem Übungsblatt weniger als 40 % in der Theorie erreichen, können Sie den praktischen Teil trotzdem bestehen – und umgekehrt.

# Theoretische Übungen

**Abgabe bis Di., 27.01., 14 Uhr**

## Allgemein

Die Lösung dieser Aufgaben werden über das Lernmodul VIPS abgegeben.

**Notiz:** Je nach Eingabe kann ein Algorithmus unterschiedlich viele Schritte benötigen. Man bezeichnet als Best Case eine Eingabe, für die der Algorithmus besonders wenig Arbeit verrichten muss, und als Worst Case eine Eingabe, für die besonders viel Arbeit anfällt.

## Aufgabe 1 – 15 Punkte

### Binäre Suche beim Einfügen (i1-u-bs0)

Gegeben sei ein sortiertes Array $A$ der Länge $n$. Ein neues Element soll so in $A$ eingefügt werden, dass das Array weiterhin sortiert bleibt.

Zur Bestimmung der Einfügeposition wird die binäre Suche verwendet.

1. Erklären Sie (kurz) informell, warum sich die Position eines neuen Elements mit binärer Suche bestimmen lässt. (3 Punkte)
2. Wie viele Vergleiche benötigt eine binäre Suche im Worst Case, um die Einfügeposition in einem Array der Länge $n$ zu finden? (4 Punkte)
3. Nachdem die Position gefunden wurde, müssen Elemente im Array verschoben werden. Wie viele Elemente müssen im Worst Case verschoben werden? (4 Punkte)
4. Geben Sie die Gesamtlaufzeit für das Einfügen eines einzelnen Elements in Tilde-Notation an. (4 Punkte)

## Aufgabe 2 – 25 Punkte

### Insertionsort (i1-u-is1)

Der Sortieralgorithmus Insertionsort sortiert ein Array, indem Elemente schrittweise an der richtigen Position eingefügt werden.

Der Algorithmus ist gegeben durch:

```
Insertionsort(A):
    for i = 1 to length(A)-1:
        key = A[i]
        j = i - 1
        while j >= 0 and A[j] > key:
            A[j+1] = A[j]
            j = j - 1
        A[j+1] = key
```

1. Führen Sie Insertionsort für das folgende Array aus:

   $$(7,\ 3,\ 4,\ 2,\ 6,\ 1,\ 8,\ 9,\ 5)$$

   Notieren Sie die Zustände des Arrays nach jedem vollständigen Durchlauf der äußeren Schleife, d.h. nach dem Einfügen von $key$.

   (10 Punkte)

2. Beschreiben Sie kurz, wie ein Best Case bzw. Worst Case für Insertionsort aussieht. (3 Punkte)

3. Wie viele Vergleiche werden im Best Case für ein Array der Länge $n$ durchgeführt? Begründen Sie Ihre Antwort kurz. Zählen Sie nur Vergleiche zwischen Arrayelementen. Angabe mit Tilde-Notation. (6 Punkte)

4. Wie viele Vergleiche werden im Worst Case für ein Array der Länge $n$ durchgeführt? Angabe mit Tilde-Notation. (6 Punkte)

## Aufgabe 3 – 10 Punkte

### Greedy-Algorithmus: Münzwechsel (i1-u-gr1)

Gegeben sei ein Münzsystem mit den Münzwerten

$$\{1,\ 2,\ 5,\ 10\}.$$

Ziel ist es, für einen gegebenen Betrag Wechselgeld mit möglichst wenigen Münzen zu erzeugen.
Ein Greedy-Algorithmus wählt dabei in jedem Schritt die größtmögliche Münze, die den verbleibenden Betrag nicht überschreitet.

1. Wenden Sie den Algorithmus an um den Betrag 29 zu wechseln. Geben Sie die gewählten Münzen in der Reihenfolge ihrer Auswahl an. (4 Punkte)

2. Betrachten Sie nun das Münzsystem

   $$\{1,\ 3,\ 4\}.$$

   Wenden Sie den Greedy-Algorithmus auf den Betrag 6 an. Vergleichen Sie das Greedy Ergebnis mit einer optimalen Lösung. Erklären Sie, in ein paar Sätzen, warum der Greedy Algorithmus keine optimale Lösung liefert. (6 Punkte)

# Praktische Übung

**Abgabe der Prüfsumme bis Di., 27.01., 14 Uhr**
**Testat ab Di., 27.01., 18 Uhr**

**Hilfe** zum Bearbeiten der praktischen Übungen können Sie grundsätzlich jeden Tag in den Rechnerübungen bekommen.

- Bearbeiten Sie die praktischen Übungen in dem bereitgestellten Jupyter Notebook.
- Um die praktische Übung testieren zu lassen, **müssen** Sie einen Termin in der zugehörigen Rechnerübung reservieren. Ein Testat ohne Termin ist nicht möglich.
- Übermitteln Sie die Prüfsumme über Vips
- Öffnen Sie die Terminvergabe für diese praktische Übung und lassen Sie den von Ihnen reservierten Termin anzeigen.
- Zeigen Sie ihr Notebook und führen Sie alle zellen aus, damit am Ende die Prüfsumme angezeigt wird.
- Der CIP-Pool befindet sich hier: `https://lageplan.uni-goettingen.de/?ident=2412_1_1.UG_-1.110`

## Aufgabe 1 – 20 Punkte

### Lévy-C-Kurve (i1-p-lvc0)

Die Lévy-C-Kurve wird zunächst mit den Zeichen `F`, `+` und `-` beschrieben.

- `+` dreht den Zeichenstift um 45° nach rechts.
- `-` dreht den Zeichenstift um 45° nach links.

Die Zeichenkette

$$c_0 = \texttt{F}$$

repräsentiert die Lévy-C-Kurve der Ordnung 0.
Ist $c_i$ die Zeichenkette der Ordnung $i$, so ist die Zeichenkette der Ordnung $i + 1$ gegeben durch

$$c_{i+1} = + \, c_i \, - - \, c_i \, + \, .$$

Zur Beschreibung der Lévy-C-Kurve wird eine Zeichenkette betrachtet, die aus den Symbolen `F` (Vorwärtsbewegung), `+` und `-` (Drehungen) besteht. Diese Zeichenkette wird schrittweise vereinfacht und in die Symbole `F`, `L` und `R` überführt.

- Zunächst werden alle Zeichen entfernt, die vor dem ersten und nach dem letzten `F` stehen, da sie keinen Einfluss auf die Kurve haben.
- Zwischen zwei aufeinanderfolgenden `F` steht stets eine gerade Anzahl von Drehzeichen (`+` und `-`).

- Befinden sich zwischen zwei `F` gleich viele `+`- und `-`-Zeichen, so heben sich diese gegenseitig auf und können gestrichen werden.
- Bleiben danach zwischen zwei `F` nur noch `-`-Zeichen übrig, dann werden je zwei `-` durch ein `L` ersetzt.
- Bleiben danach zwischen zwei `F` nur noch `+`-Zeichen übrig, dann werden je zwei `+` durch ein `R` ersetzt.

Beispiele:

| | | |
|---|---|---|
| Ordnung 0: | `F` | |
| Ordnung 1: | `+F--F+` | → `FLF` |
| Ordnung 2: | `++F--F+--+F--F++` | → `FLFFLF` |
| Ordnung 3: | `+++F--F+--+F--F++--++F--F+--+F--F+++` | → `FLFFLFRFLFFLF` |

1. Schreiben Sie eine Funktion

   `levy_next(curve)`

   die eine Lévy-C-Kurve der Ordnung $i$ als Liste der Zeichen `F`, `+`, `-` übergeben bekommt und eine neue Liste zurückliefert, die die Lévy-C-Kurve der Ordnung $i + 1$ repräsentiert. (5 Punkte)

2. Schreiben Sie eine Funktion

   `levy_flr(curve)`

   die eine Lévy-C-Kurve mit den Zeichen `F`, `+`, `-` so verändert, dass sie nur noch aus den Zeichen `F`, `L` und `R` besteht. Die Funktion soll die neue Zeichenkette zurückgeben. (5 Punkte)

3. Schreiben Sie ein Programm, das
   - die Lévy-C-Kurve der Ordnung $k$ ($0 \leq k \leq 15$) konstruiert,
   - und die Zeichenkette selbst in eine Textdatei schreibt.

   (5 Punkte)

4. Lesen Sie die erzeugte Datei wieder ein und zeichnen Sie die Lévy-C-Kurve mit `jupyturtle`. Interpretieren Sie dabei:
   - `F` als Vorwärtsbewegung,
   - `L` als Drehung um 90° nach links,
   - `R` als Drehung um 90° nach rechts.

   (5 Punkte)

## Aufgabe 2 – 30 Punkte

### Josephus-Permutation (i1-p-jp0)

Eine Permutation der Zahlen $1, \ldots, N$ ist eine beliebige Anordnung dieser Zahlen ohne Wiederholungen und Auslassungen.

Beispiel:

$$4\ 1\ 3\ 2$$

ist eine Permutation der Zahlen

$$1\ 2\ 3\ 4.$$

Eine Josephus-Permutation der Länge $N$ ist eine Permutation der Zahlen $1, \ldots, N$, für deren Bildung zusätzlich eine Schrittweite $S$ gegeben ist.

Die Josephus-Permutation wird wie folgt erzeugt:

- Die Zahlen $1, \ldots, N$ werden in einem Kreis angeordnet.
- Die Position der Zahl 1 ist die aktuelle Position.
- Solange der Kreis nicht leer ist, wird rundenweise jeweils die nächste Zahl entfernt, die $S$ Schritte von der aktuellen Position entfernt ist. Die aktuelle Position wird dabei mitgezählt.
- Die Josephus-Permutation entsteht, indem an die Position einer Zahl $k$ (in der ursprünglichen Anordnung $1 \ldots N$) die Runde eingetragen wird, in der diese Zahl aus dem Kreis entfernt wurde.

Beispiel für $N = 4$ und $S = 2$:

| Runde | Kreis |
|---|---|
| Init | [1 2 3 4] |
| 1 | [1 $x$ 3 4] |
| 2 | [1 3 $x$] |
| 3 | [1 $x$] |
| 4 | [$x$] |

Die zugehörige Josephus-Permutation lautet:

$$4\ 1\ 3\ 2$$

1. Schreiben Sie eine Funktion

   `josephus_permutation(length, step)`

   die die Länge $N$ und die Schrittweite $S$ erhält und eine Liste der Länge $N$ zurückgibt, welche die Josephus-Permutation enthält.

2. Die Funktion `josephus_permutation` soll eine Liste `circle` initialisieren, deren Einträge zunächst alle einen geeigneten Startwert $s$ enthalten, wobei $s < 1$ oder $s > N$ gilt.

3. Die eigentliche Berechnung soll durch eine rekursive Funktion

   ```
   josephus_recursive(circle, current, round, step)
   ```

   erfolgen, welche die Liste `circle` schrittweise manipuliert.

4. Es gilt:
   - Das Listenelement `circle[k]` – nicht der gespeicherte Wert – repräsentiert die Zahl $k + 1$.
   - Ist der Wert von `circle[k]` gleich dem Startwert, so befindet sich die Zahl $k+1$ noch im Kreis.
   - Andernfalls gibt `circle[k]` an, in welcher Runde die Zahl $k + 1$ aus dem Kreis entfernt wurde.
   - Eine Zahl wird aus dem Kreis entfernt, indem der entsprechende Listeneintrag mit der aktuellen Rundenzahl überschrieben wird.

Die Applikation soll abschließend die Anordnung

$$1\ 2\ \ldots\ N$$

und die zugehörige Josephus-Permutation untereinander ausgeben.
(30 Punkte)